cole.engbrecht@gmail.com
https://coleengbrecht.github.io

Coleman Engbrecht - Systems & Graphics Programmer

Engine programmer with 10+ years of experience building rendering, physics, networking, and multithreaded systems in C++ and JavaScript. My professional experience includes console development, mobile/web development, and optimizing low-latency systems

Skills:

C++, JavaScript, Dart, C#, Vulkan/OpenGL/DirectX, GLSL/HLSL, Console APIs, Linux, Unreal, Unity, Git, Perforce, Flutter, React Native

Experience:

Engine & Gameplay Programmer, Hypersect 2025 - Present

- Ported a networked multiplayer game to Amazon's Luna platform. Developed a low-latency, mobile-first web app controller that enables instant play from QR code
- Developed zero-allocation backend technology for HTTP and WebSocket communication. Implemented multiple network protocols from spec, including Protocol Buffers
- Ported core components of an in-house game engine to console platforms, including writing, optimizing and debugging console graphics API backends
- Implemented CPU and GPU optimizations resulting in up to 50% lower frame times including: tile-based light culling, reduced CPU to GPU data streaming, multithreading complex gameplay systems
- Debugged and fixed thread-scheduling related latency issues on multiple platforms, made use of ETW and console debugging tools
- Implemented a concurrent asset streaming system used to stream in large audio files, as well as a system for making sure the audio timeline doesn't de-sync with the game timeline when streaming takes longer than expected
- Wrote a physically based animation system for characters and weapons that enables dynamic, reactive animations in gameplay without needing artist authored animations
- Designed and implemented a robust save version upgrade system. It uses rule-based graph rewrites on the save data to ensure forward compatibility when schema data changes
- Improved the performance and correctness of a novel heuristic based 3D sprite sorting algorithm by: eliminating non-determinism, utilizing a spatial acceleration structure, introducing static and dynamic passes in order to improve temporal stability under camera rotation
- Built a networked vehicle system using Unreal's Chaos physics. Improved the feel of a networked character controller in Unreal including: soft character vs character collision, gamepad haptics, input buffering for jumps and sprinting, heuristic driven first-person item interaction

Software Developer, gskinner 2019 - 2025

- Code ownership of a Pixi JS based web app. Re-wrote the input manager to provide better support for pen devices on Android, implemented an advanced gesture detection system using the custom input manager. Made significant architectural changes that lead to 10x performance improvements and removed garbage collection stutters. Wrote libraries and APIs that were used across multiple apps leading to similar improvements
- Was the go-to engineer for writing shaders and GPU code for multiple projects. Gave technical talks and training to peers on shader programming and relevant math. Implemented GPU rendering features in Unity and Flutter
- Integrated deep support for the Zoom SDK utilizing CallKit and other native iOS APIs into a finance app built in Flutter. Implemented support for answering VoIP call notifications in app and displaying video

through Flutter's native views
- Built multiple cross platform OS integration plugins for Flutter utilizing C++, Swift, Obj-C, Kotlin, and Java including: custom window decorations and controls for desktop platforms, firebase SDK integration
- Wrote CI/CD pipelines that tested, packaged and published software for many platforms including: Snap Store, Windows Store, App Store, Google Play and Github
- Communicated with clients to fine tune technical requirements and implementation details for deliverables

Game Engine Development, Oak Engine 2015 - Present

I designed and developed a cross platform game engine from scratch over a period of 10 years. The engine has a minimal runtime footprint, relying only on SDL for input/window management and STB for vorbis decoding. I've used this engine to publish multiple small games and develop internal tools that I use on a daily basis for my professional work.

- Implemented a connection oriented network protocol that supports channel multiplexing with different reliability and ordering behavior
- Implemented a reflection and serialization framework that enables: scene serialization, automatic symmetric diffing for undo/redo, network message serialization, real-time game state replication with record and replay functionality, property editing GUI
- Implemented a simple and fast entity component system with efficient cache behavior, support for entity links and transform hierarchies
- Implemented a low latency job system utilizing: atomic operations, futexes, x86-64 and aarch64 fibers, lock-free data structures. On top of this job system I've built: an async asset loading system with async GPU data upload, shadow and light culling system, physically based particle simulation
- Used address and undefined behavior sanitizers to detect and fix correctness issues in low-level context switching code, memory allocators and other engine systems
- Implemented 2D and 3D rigid body physics engines with: island solvers, SAT and GJK collision detection, robust contact manifold generation, dual quaternion mechanics, sub-stepping solvers, soft constraints, warm starting
- Implemented dual quaternion based skeletal animation system with: animation blending, blend spaces, full body IK, joint attachments
- Wrote a 3D, physically based, GPU driven renderer that supports fully dynamic direct and indirect lighting, post processing and tone-mapping. Ray tracing is implemented using a custom software voxel ray tracer that scales down to low end hardware. In engine path tracer that can be used as a reference for real-time approximations. It supports modern BRDFs such as GGX and Disney-Diffuse
- Implemented a GPU compute particle system that can simulate and render 5+ million particles in real time using indirect draw commands, wave intrinsics, atomics and GPU-native parallel algorithms. It supports per-particle culling and skinned particles for foliage and soft bodies

Aircraft De-icing Technician, AeroMag 2017 - 2019

- Communicated effectively over the radio in a safety critical environment

Education:

Computer Engineering (CNT), North Alberta Institute of Technology 2018-2019